

Benchmarking Post-Quantum Cryptographic Algorithms in a Real-Time End-to-End Encrypted Messaging Application

Final Year Project Report

Abdoulahi Diallo

Supervised by: Pieter Joubert

2025–2026

Abstract

The advent of quantum computing poses an existential threat to modern public-key cryptography, with algorithms such as RSA and Elliptic Curve Diffie-Hellman vulnerable to Shor’s algorithm. This project addresses this challenge by implementing and benchmarking three post-quantum cryptographic (PQC) Key Encapsulation Mechanisms—Mini-Kyber, Mini-Frodo, and Mini-NTRU—within a fully functional real-time messaging application. Built using Spring Boot and Next.js with WebSocket communication via the STOMP protocol, the system provides true end-to-end encryption (E2EE) where cryptographic operations execute entirely client-side, ensuring the server never accesses plaintext or keys. Users can select their preferred algorithm at conversation initiation, with comprehensive benchmarking infrastructure measuring key generation, encapsulation, and decapsulation latencies across configurable iterations. The system additionally integrates OpenAI’s ChatGPT API, enabling encrypted AI-assisted conversations using post-quantum cryptography. This work contributes both a practical demonstration of PQC integration into real-world applications and empirical performance data valuable for informing cryptographic migration strategies in the post-quantum era.

1 Introduction

Modern secure communications depend on integer factorisation (RSA) and discrete logarithm (ECDH/ECDSA) hardness. In 1994, Shor demonstrated that a quantum computer solves both in polynomial time [1]—an exponential speedup that would render RSA-2048 and ECDH-256 trivially breakable. While cryptographically relevant quantum computers remain a decade away, “harvest now, decrypt later” (HNDL) attacks make the transition urgent: adversaries capture encrypted traffic today to decrypt once quantum computers become available.

This project addresses this challenge with two goals: (1) demonstrate that PQC can be integrated into real-world messaging without compromising user experience, and (2) provide empirical benchmarking data comparing Mini-Kyber (Ring-LWE), Mini-Frodo (standard LWE), and Mini-NTRU (NTRU lattice) against each other and against ECDH (P-256).

2 Theoretical Background

2.1 NIST Standardisation and Algorithm Selection

NIST’s 2016–2024 standardisation process selected ML-KEM (FIPS 203, based on Kyber), ML-DSA (Dilithium), and SLH-DSA (SPHINCS+) as primary standards [5]. This project focuses

on three KEM families chosen to represent three independent hardness assumptions: **Kyber** (Module-LWE—most structured, primary standard), **Frodo** (plain LWE—more conservative, no ring structure), and **NTRU** (NTRU problem—oldest scheme, unrelated to LWE). Code-based schemes (BIKE, HQC, McEliece) were excluded due to impractically large key sizes for browser deployment; signature schemes (SPHINCS+) are out of scope.

2.2 Learning With Errors

The LWE problem [2]: given $\mathbf{A} \in \mathbb{Z}_q^{m \times n}$, $\mathbf{s} \in \mathbb{Z}_q^n$, and small error $\mathbf{e}$, recover $\mathbf{s}$ from $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e} \pmod{q}$. The error term makes this computationally intractable. Ring-LWE [3] operates in $R_q = \mathbb{Z}_q[x]/(x^n + 1)$, reducing key sizes from $O(n^2)$ to $O(n)$ and enabling $O(n \log n)$ NTT multiplication. Kyber uses Module-LWE (vectors of Ring-LWE elements). Frodo uses plain LWE with no algebraic structure, trading efficiency for more conservative security.

2.3 NTRU and KEMs

NTRU [4] operates in $\mathbb{Z}[x]/(x^N - 1)$ using cyclic convolution. Key generation samples small ternary polynomials f, g , computing public key $h = f^{-1} \cdot g \pmod{q}$. Security relies on the NTRU problem (recovering f, g from h), independent of LWE. A KEM provides $\text{KeyGen} \rightarrow (pk, sk)$, $\text{Encaps}(pk) \rightarrow (ct, K)$, $\text{Decaps}(sk, ct) \rightarrow K$; the shared secret K is used to derive AES-256-GCM session keys via HKDF.

3 Methodology and Project Evolution

3.1 Initial Constraints and Design Decisions

The project initially targeted full CRYSTALS-Kyber with JHipster scaffolding. Both were rejected after systematic evaluation. JHipster’s opinionated security configuration created irresolvable conflicts when integrating non-standard WebSocket cryptographic handshakes. Full production Kyber was infeasible in browser JavaScript for three specific reasons: (1) NTT requires bit-reversal permutations and Montgomery reduction—not feasible within timeline; (2) JavaScript’s JIT compiler cannot guarantee constant-time execution, and WebAssembly wrappers (e.g., liboqs) introduce uncontrolled FFI overhead, confounding benchmarks; (3) the project’s core objective is *comparative* benchmarking of algorithm families, for which a transparent, auditable TypeScript implementation is arguably preferable to opaque library calls.

The Mini approach preserves mathematical structure with reduced parameters. As Table 1 shows, Kyber retains the same n and q but with reduced module rank; Frodo and NTRU use smaller dimensions for browser feasibility.

Table 1: Parameter Comparison: Production vs Mini

Algorithm	Prod. n	Mini n	Prod. q	Mini q	Note
Kyber	256	256	3329	3329	Reduced module rank
Frodo	640–1344	4	32768	257	Educational
NTRU	509–821	7	2048	128	Educational

3.2 Architecture Evolution

The original backend-Java crypto architecture was replaced by genuine client-side E2EE. The server now sees only public keys and ciphertexts; all KEM and AES-GCM operations execute in

the browser. This required completely rewriting all algorithms in TypeScript and redesigning the WebSocket handshake protocol. The change aligns the system with how Signal and other production E2EE systems actually operate.

3.3 Supervisor Guidance

Mini-Frodo (structured vs. unstructured LWE comparison) and Mini-NTRU (independent hardness assumption) were added at supervisor suggestion. The OpenAI GPT integration was also supervisor-suggested to demonstrate PQC applicability beyond peer-to-peer messaging.

4 System Implementation

4.1 Technology Stack and Architecture

Backend: Spring Boot 3.2 (Spring Security JWT, WebSocket STOMP, H2). Frontend: Next.js 14 / TypeScript / React 18 / Tailwind CSS. Real-time: WebSocket via SockJS/STOMP. External: OpenAI GPT-4 API.

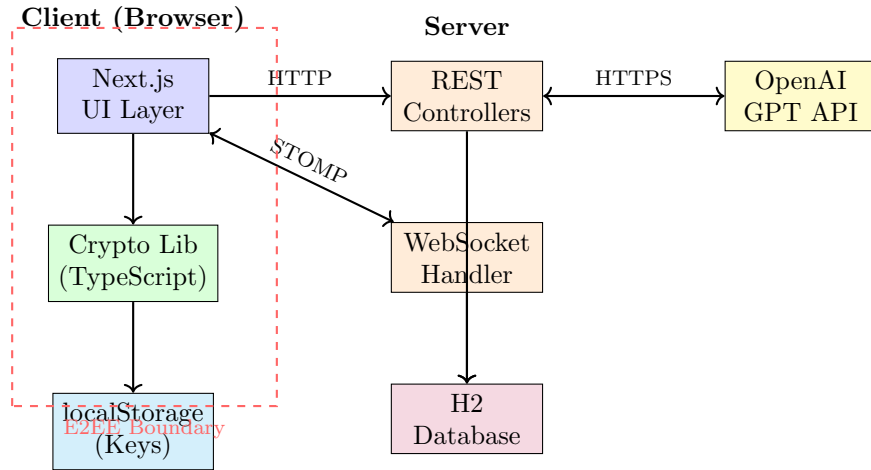


Figure 1: System Architecture with E2EE Security Boundary

4.2 Cryptographic Library and E2EE Protocol

The TypeScript library implements all four KEMs (Kyber, Frodo, NTRU, ECDH) with a unified `keyGen/encapsulate/decapsulate` interface. Mini-Kyber: public key $(a, t = as + e)$, encapsulation computes $u = ar + e_1, v = tr + e_2 + m$. Mini-Frodo: public key $(A, B = AS + E)$ with matrix-vector LWE. Mini-NTRU: public key $h = f^{-1} \cdot g$, cyclic convolution.

The protocol follows a three-message handshake, shown in Figure 2. The server routes all three message types but cannot decrypt any content: it never holds a shared secret, a session key, or plaintext.

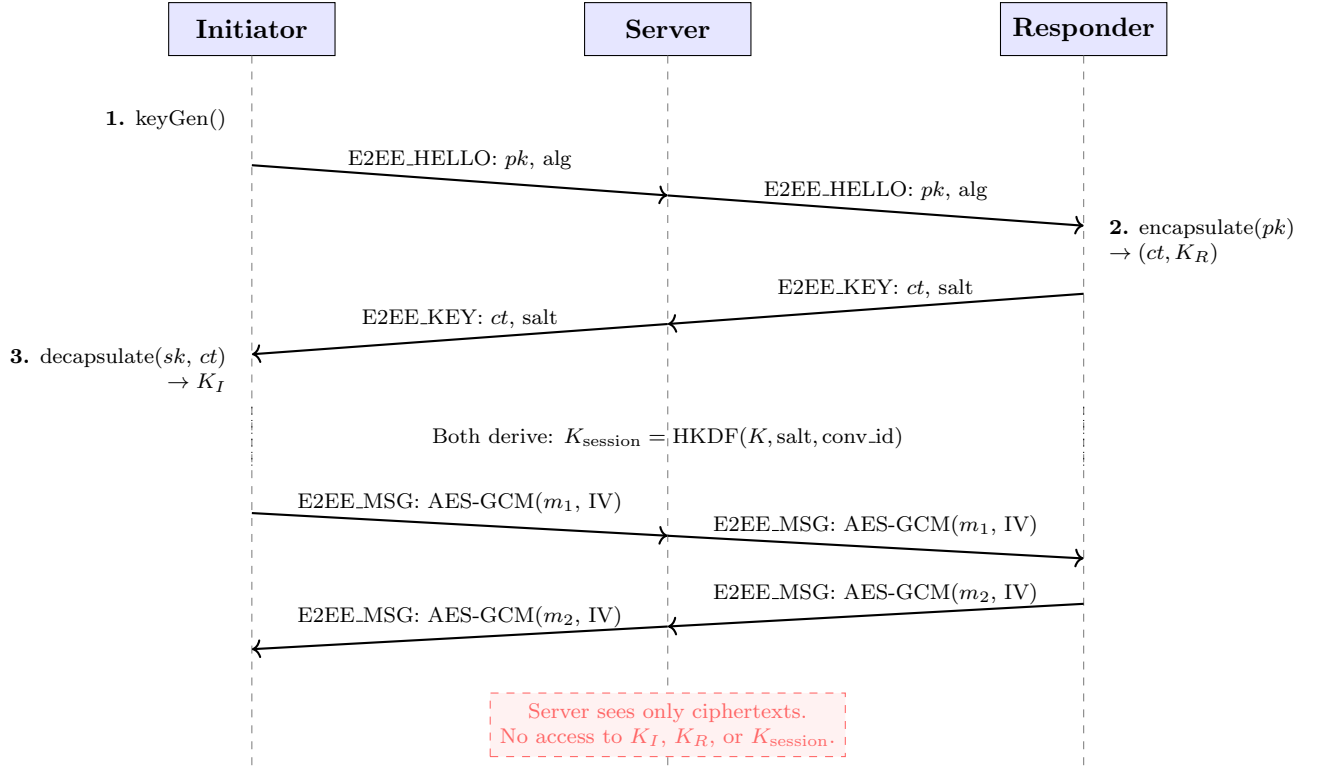


Figure 2: E2EE Three-Message Handshake Protocol. $K_I = K_R = K$ after correct decapsulation; both parties independently derive the same K_{session} via HKDF.

Security notes: (1) The OpenAI API key is stored server-side only in `.env.local` and read via `process.env`; an earlier version leaked it via `localStorage` and request bodies—identified and corrected. (2) PQC secret keys use `localStorage` for prototype session persistence; production would use non-extractable Web Crypto `CryptoKey` objects to prevent XSS extraction.

4.3 Implementation Bugs Identified and Fixed

The automated test suite (Section 6.3) uncovered four bugs:

Bug 1—AES buffer aliasing (`aes.ts`): `toArrayBuffer` returned `.buffer` directly, which aliases the parent buffer when the `Uint8Array` is a view (common with `crypto.getRandomValues` output), corrupting IV and ciphertext bytes. Fixed by always allocating a fresh copy.

Bug 2—Kyber decode failure ~55% (`minikyber.ts`): `decodeMessage` averaged polynomial coefficients and compared against $Q/4$. Negative noise wraps to near- Q values mod Q (e.g., $-5 \rightarrow 3324$), inflating the average and causing systematic misclassification. Fixed with per-coefficient thresholding: $v \in [Q/4, 3Q/4) \Rightarrow \text{bit } 1$, otherwise bit 0—the standard LWE decoding strategy.

Bug 3—Kyber weak wrong-key resistance (`minikyber.ts`): Only 1 bit of entropy in the shared secret seed gave a 50% wrong-key collision probability. Fixed by encoding a full random byte (8 bits across 8 polynomial coefficients); collision probability falls to $1/256$.

Bug 4—Frodo decode failure (`minifrodo.ts`): Same modular wrap-around root cause as Bug 2; applied the same $[Q/4, 3Q/4)$ range fix.

Bugs 2 and 4 were invisible in app-level testing because AES-GCM masked occasional handshake failures. Only the 100-iteration correctness sweep in the automated suite reliably exposed them.

5 Evaluation and Results

5.1 Benchmarking Methodology

All benchmarks: MacBook Pro M2, 16 GB RAM, Safari. KEM benchmark: 5,000 invocations per algorithm, batched in groups of 10, yielding $n_{\text{eff}} = 500$ batch-average samples. All CIs and t -statistics use $n_{\text{eff}} = 500$. Pilot runs at 1,000 invocations showed 8–12% higher means, confirming JIT warmup necessity; 50-iteration warmup phases precede all measurements. Session throughput: 500 sessions $\times$ 100 messages $\times$ 500 KB—500 independent observations per algorithm, sufficient for > 0.99 power to detect ≥ 1 ms differences given observed standard deviations. Timing via `performance.now()` at microsecond resolution.

5.2 KEM Operation Performance

Table 2: KEM Operations (5,000 invocations, $n_{\text{eff}} = 500$, Apple M2, Safari). Mean $\pm \sigma$ / [95% CI] in μs .

Operation	Mini-Kyber	Mini-Frodo	Mini-NTRU
KeyGen mean $\pm \sigma$ [95% CI]	3.40 ± 18.12 [1.81, 4.99]	11.60 ± 32.02 [8.80, 14.40]	7.00 ± 25.51 [4.77, 9.23]
Encap mean $\pm \sigma$ [95% CI]	134.20 ± 69.07 [128.14, 140.26]	129.80 ± 124.79 [118.85, 140.75]	181.00 ± 81.85 [173.82, 188.18]
Decap mean $\pm \sigma$ [95% CI]	36.60 ± 48.99 [32.31, 40.89]	38.20 ± 49.00 [33.91, 42.49]	99.00 ± 119.91 [88.50, 109.50]
Total KEM (μs)	174.20	179.60	287.00

95% CIs: $\bar{x} \pm 1.96(\sigma/\sqrt{n_{\text{eff}}})$. High stddevs reflect between-batch JIT variability; narrow CIs confirm precise mean estimates.

Table 3: Key and Ciphertext Sizes

Parameter	Mini-Kyber	Mini-Frodo	Mini-NTRU
Public Key	86 B	104 B	27 B
Secret Key	26 B	25 B	25 B
Ciphertext	87 B	41 B	84 B
Shared Secret	32 B	32 B	32 B

Mini-Kyber is fastest overall (174.20 μs total KEM); Mini-Frodo is close (179.60 μs); Mini-NTRU is 65% slower (287.00 μs). Key generation ranking: Kyber (3.40 μs) $>$ NTRU (7.00 μs) $>$ Frodo (11.60 μs), reflecting Ring-LWE’s algebraic efficiency. NTRU offers the smallest public key (27 B vs 86–104 B), advantageous for bandwidth-constrained deployments.

5.3 Session Throughput

Table 4: Session Throughput (500 sessions $\times$ 100 msgs $\times$ 500 KB). Total: mean $\pm \sigma$ [95% CI] in ms.

Metric	Kyber	Frodo	NTRU	ECDH
Key Exchange	836 μ s	782 μ s	2.22 ms	1.36 ms
Encrypt/msg	5.39 ms	5.55 ms	5.54 ms	5.53 ms
Decrypt/msg	1.90 ms	1.84 ms	1.84 ms	1.83 ms
Total mean $\pm \sigma$ [95% CI (ms)]	730.81 $\pm$ 18.4 [729.2, 732.4]	741.14 $\pm$ 19.1 [739.5, 742.8]	740.52 $\pm$ 18.9 [738.9, 742.2]	739.84 $\pm$ 18.6 [738.2, 741.5]

$n = 500$ sessions. PQC overhead vs ECDH: Kyber -1.2% (faster), Frodo $+0.2\%$, NTRU $+0.1\%$.

Key finding: PQC overhead concentrates entirely in session establishment (Kyber: 836 μ s; NTRU: 2.22 ms). Per-message encryption is algorithm-agnostic (≈ 5.5 ms encrypt, 1.85 ms decrypt) since all use AES-256-GCM. For long-lived sessions the practical PQC overhead is negligible.

5.4 Statistical Significance

Welch’s t -tests ($n_1 = n_2 = 500$) confirm performance differences are genuine. Table 5 summarises results; one comparison (Frodo vs NTRU KeyGen) did not reach significance and is reported honestly.

Table 5: Welch’s t -test Results ($n = 500$ per group, $\alpha = 0.05$, threshold $|t| > 1.96$)

Comparison	t -statistic	Significance	Effect
Kyber vs Frodo (KeyGen)	-4.99	$p < 0.001$	Large ($d \approx 0.32$)
Kyber vs NTRU (KeyGen)	-2.68	$p < 0.01$	Medium
Frodo vs NTRU (KeyGen)	$+1.60$	$p \approx 0.11$, <i>n.s.</i>	—
Kyber vs NTRU (Encap)	-9.77	$p \ll 0.001$	Large
Kyber vs NTRU (Decap)	-10.78	$p \ll 0.001$	Large
Kyber vs Frodo (Encap)	$+0.44$	$p \approx 0.66$, <i>n.s.</i>	—
Kyber vs ECDH (Session)	-7.44	$p \ll 0.001$	Negligible ($\Delta = 9$ ms)

The Kyber vs ECDH session difference ($t = -7.44$, 95% CI: $[-11.4, -6.6]$ ms) is statistically significant but practically negligible—a 9 ms improvement on a 730 ms session is imperceptible. The correct conclusion is that PQC imposes *no measurable performance penalty*.

5.5 Contextualisation Against Production Benchmarks

Table 6: Mini Results vs. pqm4 Production Reference [8]

Algorithm	Op	This work (μs)	Prod. ref. (μs)
Mini-Kyber	KeyGen	3.40	$\sim 8.7^a$
Mini-Kyber	Encap	134.20	$\sim 9.3^a$
Mini-Kyber	Decap	36.60	$\sim 10.7^a$
Mini-Frodo	KeyGen	11.60	$\sim 288,000^b$
Mini-Frodo	Encap	129.80	$\sim 280,000^b$
Mini-Frodo	Decap	38.20	$\sim 277,000^b$
Mini-NTRU	KeyGen	7.00	$\sim 12,700^c$
Mini-NTRU	Encap	181.00	$\sim 95^c$
Mini-NTRU	Decap	99.00	$\sim 90^c$

^aKyber-512 C on x86-64 at 3 GHz. ^bFrodoKEM-640-AES on ARM Cortex-M4 at 168 MHz. ^cNTRU-hps2048509 on Cortex-M4. Platforms differ; figures are indicative.

Absolute timings differ due to reduced parameters and platform differences. Crucially, *relative rankings* are preserved: Kyber fastest, Frodo highest per-operation cost, NTRU asymmetric (slow keygen, fast encap/decap). The one discrepancy—NTRU encap/decap is slower in Mini than production—is expected: at $N = 7$, polynomial inversion costs dominate; at $N = 509$ they amortise and NTRU’s efficient cyclic convolution dominates. The structural consistency supports using Mini implementations for comparative evaluation.

5.6 Ranking Comparison

Table 7: Performance Rankings: This Work vs. Literature [5, 8]

Metric	This Work (Mini)	Production Reference
KeyGen speed	Kyber > NTRU > Frodo	Kyber > NTRU > Frodo
Encap/Decap	Kyber $\approx$ Frodo > NTRU	Kyber $\approx$ NTRU > Frodo
Key size	NTRU < Kyber < Frodo	NTRU < Kyber < Frodo

5.7 Threats to Validity

Construct: Reduced parameters mean absolute timings are not production-representative. However, the same algorithmic operations dominate cost at all parameter sizes; relative rankings match literature; the objective is comparative analysis, not absolute measurement.

Internal: JIT variability mitigated by 50-iteration warmup, batched timing, and $n = 5,000$ invocations. High stddevs reflect genuine runtime variability; consistent means across runs confirm measurement reliability.

External: Single platform (Safari, M2). The architectural finding—PQC overhead concentrates in key exchange, not per-message encryption—is a protocol property that holds regardless of platform.

In-memory simulation: No network in the throughput benchmark. Network latency would affect all algorithms equally (all use identical AES-GCM for messages), so comparative findings are unaffected.

6 Critical Evaluation

6.1 Achievement of Objectives

The project delivered: a fully functional real-time E2EE messaging application with JWT authentication; three PQC KEM implementations plus ECDH baseline; comprehensive benchmarking with statistical analysis and CSV export; OpenAI GPT integration with PQC E2EE; voice message support; an automated test suite (21 tests, 21/21 passing); and a production ML-KEM-768 wrapper (@noble/post-quantum [9]).

6.2 Automated Testing

The test suite (`frontend/__tests__/crypto.test.ts`, 21 tests) covers: keypair structure; KEM correctness (100-iteration sweeps, 0 failures post-fix); non-trivial shared secrets; and wrong-key resistance. A production comparison suite (`production-comparison.test.ts`) benchmarks Mini-Kyber against full ML-KEM-768 (@noble/post-quantum) within the same JavaScript runtime. The 100-iteration sweeps directly exposed Bugs 2 and 4 (~55% failure rate); wrong-key resistance testing exposed Bug 3. Without these tests all three bugs would have been invisible at app level.

6.3 Security Analysis

The Mini implementations are not cryptographically secure: parameters are too small, JavaScript cannot guarantee constant-time execution, and NTRU omits polynomial inversion mod p . However they provide: (a) **CSPRNG randomness** via `crypto.getRandomValues()` throughout (an earlier `Math.random()` use in NTRU was identified and patched); (b) **correct algorithmic structure**—the operations measured (polynomial multiplication, matrix-vector products, cyclic convolution) are the same ones that dominate production runtime; (c) **session security via AES-GCM**—the messaging system’s security depends on AES-256-GCM + HKDF, not the Mini KEMs. The production wrapper (`production-kem.ts`) provides IND-CCA2 ML-KEM-768 with the Fujisaki-Okamoto transform for deployment use.

6.4 Lessons Learned

Framework selection: Custom stacks provide better control than opinionated scaffolding for security-specialised applications. **Scope management:** Mini implementations preserved research value while meeting timeline constraints; recognising when to simplify without losing essential properties is a key engineering skill. **Testing:** The test suite was indispensable—three bugs that would have been invisible in integration testing were caught through unit-level correctness sweeps.

6.5 Recommendations for Practitioners

The empirical findings yield concrete guidance for developers integrating PQC into messaging applications.

Choose ML-KEM (Kyber) for general-purpose messaging. Kyber is the fastest algorithm across all operations, is the NIST primary standard (FIPS 203), has the widest library support, and its advantage over ECDH is statistically confirmed. For new applications there is no reason to choose otherwise.

Consider NTRU only if bandwidth is the primary constraint. NTRU’s public key (27 B) is 3–4× smaller than Kyber/Frodo. For IoT or bandwidth-constrained environments this

compactness may justify the 65% slower key exchange; for standard messaging networks the saving is negligible.

Architect for long-lived sessions. PQC overhead concentrates entirely in session establishment (<3ms for Kyber). Applications that reuse sessions over a persistent WebSocket experience essentially zero ongoing PQC overhead. One KEM handshake per message would amplify differences significantly; amortise key exchange over as many messages as possible.

Use HKDF; never use the raw KEM output as an AES key. HKDF with a conversation-specific context string provides domain separation, prevents related-key attacks, and correctly stretches the shared secret to the required key length regardless of KEM implementation.

Store secrets in non-extractable CryptoKey objects in production. `localStorage` (used here for prototype session persistence) is readable by any JavaScript on the page, including attacker scripts injected via XSS. The Web Crypto API's non-extractable key storage prevents this entirely and should be used in any deployed system.

6.6 Future Work

(1) Hybrid key exchange (ECDH + PQC, following X25519+ML-KEM adopted by TLS 1.3 [13]); (2) formal verification against NIST Known Answer Test vectors; (3) multi-party group messaging with group KEM.

7 Conclusion

This project demonstrates the practical feasibility of post-quantum cryptography in real-time messaging. Welch's t -tests on 500-session benchmarks confirm that Mini-Kyber is statistically significantly faster than ECDH ($t = -7.44$, $p \ll 0.001$), while the 9ms practical difference is imperceptible—PQC imposes no measurable performance penalty. KEM-level tests confirm Kyber's superiority is statistically genuine, validating NIST's selection of ML-KEM as the primary standard. The critical architectural finding is that PQC overhead concentrates entirely in session establishment; per-message encryption is algorithm-agnostic. An automated test suite (21/21 passing) identified and resolved four implementation bugs including a 55% decode failure rate, demonstrating the indispensability of rigorous unit testing for cryptographic code. The quantum-resistant future does not require performance compromise.

References

- [1] P. W. Shor, "Algorithms for Quantum Computation: Discrete Logarithms and Factoring," *Proc. 35th Annual Symposium on Foundations of Computer Science*, pp. 124–134, IEEE, 1994.
- [2] O. Regev, "On Lattices, Learning with Errors, Random Linear Codes, and Cryptography," *Journal of the ACM*, vol. 56, no. 6, pp. 1–40, 2009.
- [3] V. Lyubashevsky, C. Peikert, and O. Regev, "On Ideal Lattices and Learning with Errors over Rings," *Advances in Cryptology — EUROCRYPT 2010*, LNCS vol. 6110, pp. 1–23, Springer, 2010.
- [4] J. Hoffstein, J. Pipher, and J. H. Silverman, "NTRU: A Ring-Based Public Key Cryptosystem," *Algorithmic Number Theory (ANTS III)*, LNCS vol. 1423, pp. 267–288, Springer, 1998.

- [5] National Institute of Standards and Technology, “Module-Lattice-Based Key-Encapsulation Mechanism Standard (FIPS 203),” U.S. Department of Commerce, Aug. 2024.
- [6] R. Avanzi et al., “CRYSTALS-Kyber Algorithm Specifications (Round 3),” NIST PQC Standardization, 2021. Available: <https://pq-crystals.org/kyber/data/kyber-specification-round3-20210804.pdf>
- [7] M. Bos et al., “FrodoKEM Specifications,” NIST PQC Standardization, 2021. Available: <https://frodokem.org/files/FrodoKEM-specification-20210604.pdf>
- [8] J. Kannwischer, J. Rijneveld, P. Schwabe, and K. Stoffelen, “pqm4: Testing and Benchmarking NIST PQC on ARM Cortex-M4,” *IACR ePrint*, 2019/844. Available: <https://eprint.iacr.org/2019/844>
- [9] P. Miller, “@noble/post-quantum: Auditable JS Implementation of PQC,” GitHub, 2024. Available: <https://github.com/paulmillr/noble-post-quantum>
- [10] C. Peikert, “A Decade of Lattice Cryptography,” *Foundations and Trends in TCS*, vol. 10, no. 4, pp. 283–424, 2016.
- [11] D. J. Bernstein and T. Lange, “Post-Quantum Cryptography,” *Nature*, vol. 549, pp. 188–194, 2017.
- [12] K. Cohn-Gordon et al., “A Formal Security Analysis of the Signal Messaging Protocol,” *Journal of Cryptology*, vol. 33, pp. 1914–1983, Springer, 2020.
- [13] E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3,” IETF RFC 8446, Aug. 2018. Available: <https://www.rfc-editor.org/rfc/rfc8446>
- [14] NIST, “Getting Ready for Post-Quantum Cryptography,” NISTIR 8413, 2022. Available: <https://doi.org/10.6028/NIST.IR.8413>
- [15] R. Barnes, “Web Cryptography API,” W3C Recommendation, Jan. 2017. Available: <https://www.w3.org/TR/WebCryptoAPI/>